IF2110 Algoritma dan Pemrograman 2

Type Bentukan

Tim Pengajar IF2110

Sekolah Teknik Elektro dan Informatika

Tujuan

- Mahasiswa mampu memahami definisi type bentukan
- Berdasarkan definisi yang dipahami, mahasiswa mampu membuat program yang memanfaatkan type bentukan
- Mahasiswa mampu mengimplementasikan type bentukan dalam Haskell

Type

- Type adalah himpunan nilai dan sekumpulan operator yang terdefinisi terhadap type tersebut
  - Dalam konteks fungsional: operator dijabarkan dalam bentuk fungsi
- Jenis-jenis type:
  - Type dasar → sudah tersedia: integer, real, character, boolean
  - Type bentukan → dibuat sendiri

Type Bentukan

- Himpunan nilai: ditentukan oleh domain nilai komponennya → bisa type dasar, bisa type bentukan
- Menyatakan nilai type bentukan: dalam bentuk tuple.
  - Contoh: nilai suatu Point didefinisikan oleh tuple <x:integer, y:integer>, misal: <0,0>
- Operator: harus dibuat definisi, spesifikasi, dan realisasinya sendiri tergantung pada operasi yang berlaku pada type bentukan

Type Bentukan

- Beberapa jenis type bentukan:
  - Type yang memberikan nama baru type lain/membatasi domain type lain
    - Contoh: type bilbulat : integer → memberi nama baru
      type IntPos : integer > 0 → membatasi domain
  - Type yang terdiri atas komponen-komponen bertype lain
    - Contoh: type Point : <x : integer, y : integer>
  - Type bentukan tanpa nama
    - Contoh: range dari fungsi HHMMDD (diktat Tipe Bentukan hlm. 10)
      HHMMDD: integer[0..99999] → <integer ≥ 0, integer[0..23],
      2 integer[0..59]>

Mendefinisikan Type

- Dalam konteks fungsional, mendefinisikan type adalah mendefinisikan:
  - Nama dan struktur type (komponen-komponennya)
  - Selektor untuk mengakses komponen-komponen type
  - Konstruktor untuk “membentuk” type
  - Predikat untuk menentukan karakteristik dan pemeriksaan besaran
  - Fungsi-fungsi lain yang didefinisikan untuk type tersebut

Type Point

- Diktat Tipe Bentukan hlm. 3-4
  - Definisi type Point
  - Definisi dan spesifikasi Selektor
  - Definisi dan spesifikasi Konstruktor
  - Definisi, spesifikasi, realisasi Predikat
    - Contoh: IsOrigin?
  - Definisi, spesifikasi, realisasi Fungsi Lain
    - Contoh: Jarak; Kuadran

TYPE POINT

DEFINISI TYPE

type point: <x: real, y: real>

{<x,y> adalah sebuah point, dengan x adalah absis, y adalah ordinat}

DEFINISI DAN SPESIFIKASI SELEKTOR

Absis: point → real

{Absis(P) memberikan absis Point P}

Ordinat: point → real

{Ordinat(P) memberikan ordinat Point P}

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

MakePoint: 2 real → point

{MakePoint(a,b) membentuk sebuah point dari a dan b dengan a sebagai absis dan b sebagai ordinat}

DEFINISI DAN SPESIFIKASI PREDIKAT

IsOrigin?: point → boolean

{IsOrigin? (P) benar jika P adalah titik origin yaitu titik <0,0>}

DEFINISI OPERATOR/FUNGSI LAIN TERHADAP POINT

Jarak: 2 point → real

{Jarak(P1,P2): menghitung jarak antara 2 point P1 dan P2}

Jarak0: point → real

{Jarak0(P1) menghitung jarak titik terhadap titik pusat koordinat (0,0)}

Kuadran: point → integer[1..4]

{Kuadran(P) menghitung kuadran dimana titik tersebut terletak. Syarat P bukan titik origin dan bukan terletak pada sumbu X dan bukan terletak pada sumbu Y}

{Fungsi antara yang dipakai: FX2 adalah pangkat dua yang pernah didefinisikan pada least square dan SQRT(X) adalah fungsi dasar untuk menghitung akar}

REALISASI

IsOrigin?(x,y): x=0 and y=0

IsOrigin?(P): Absis(P)=0 and Ordinat(P)=0

Jarak 2 titik= $\sqrt{((x_1-x_2)^2 + (y_1-y_2)^2)}$

Jarak(P1,P2):
    SQRT (FX2 (Absis(P1) – Absis(P2)) +
            FX2 (Ordinat(P1) – Ordinat(P2))
)

Jarak dgn titik 0 = $\sqrt{(x^2 + y^2)}$

Jarak0(P):
    SQRT (FX2 (Absis(P1)) + FX2 (Ordinat(P1)))

Kuadran(P):
    depend on Absis(P), Ordinat(P)
        Absis(P) > 0 and Ordinat(P) > 0: 1
        Absis(P) < 0 and Ordinat(P) > 0: 2
        Absis(P) < 0 and Ordinat(P) < 0: 3
        Absis(P) > 0 and Ordinat(P) > 0: 4

Kuadran(x,y):
    depend on x,y
        x > 0 and y > 0: 1
        x < 0 and y > 0: 2
        x < 0 and y < 0: 3
        x > 0 and y > 0: 4

Type Pecahan

- Diktat Tipe Bentukan hlm. 4-5
  - Definisi type Pecahan
  - Definisi dan spesifikasi Selektor
  - Definisi dan spesifikasi Konstruktor
  - Definisi, spesifikasi, realisasi Predikat
    - Contoh: IsEqP?; IsLtP?
  - Definisi, spesifikasi, realisasi Fungsi Lain
    - Contoh: AddP; DivP

TYPE PECAHAN

DEFINISI DAN SPESIFIKASI TYPE

type pecahan: <n: integer ≥ 0, d: integer > 0>

{<n:integer ≥ 0, d:integer > 0>: n adalah pembilang (numerator) dan d adalah penyebut (denumerator). Penyebut sebuah pecahan tidak boleh nol}

DEFINISI DAN SPESIFIKASI SELEKTOR

Pemb: pecahan → integer ≥ 0

*{Pemb(p) memberikan numerator pembilang **n** dari pecahan tsb}*

Peny: pecahan → integer > 0

*{Peny(p) memberikan denumerator penyebut **d** dari pecahan tsb}*

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

MakeP: integer ≥ 0, integer > 0 → pecahan

{MakeP(x,y) membentuk sebuah pecahan dari pembilang x dan penyebut y dengan x dan y integer}

DEFINISI DAN SPESIFIKASI OPERATOR TERHADAP PECAHAN

{Operator Aritmatika Pecahan}

AddP: 2 pecahan → pecahan
{AddP(P1,P2): menambahkan dua buah pecahan P1 dan P2:
*n1/d1 + n2/d2 = (n1*d2 + n2*d1) / (d1*d2)}*

SubP: 2 pecahan → pecahan
{SubP(P1,P2): mengurangkan dua buah pecahan P1 dan P2:
*n1/d1 - n2/d2 = (n1*d2 - n2*d1) / (d1*d2)}*

MulP: 2 pecahan → pecahan
{MulP(P1,P2): mengalikan dua buah pecahan P1 dan P2:
*(n1/d1) * (n2/d2) = (n1*n2) / (d1*d2)}*

DivP: 2 pecahan → pecahan
{DivP(P1,P2): membagi dua buah pecahan P1 dan P2, dengan syarat
Pemb(P2) > 0 (pecahan pembilang bukan 0):
*(n1/d1) / (n2/d2) = (n1*d2) / (d1*n2)}*

RealP: pecahan → real
{Menuliskan pecahan dalam notasi desimal}

DEFINISI DAN SPESIFIKASI PREDIKAT

{Operator relasional Pecahan}

IsEqP?: 2 pecahan → boolean
{IsEqP?(P1,P2) true jika P1=P2
membandingkan 2 pecahan sama nilainya n1/d1 = n2/d2
*jika dan hanya jika n1*d2 = n2*d1}*

IsLtP?: 2 pecahan → boolean
{IsLtP?(P1,P2) true jika P1<P2
membandingkan 2 pecahan, apakah P1 < P2: n1/d1 < n2/d2
*jika dan hanya jika n1*d2 < n2*d1}*

IsGtP?: 2 pecahan → boolean
{IsGtP?(P1,P2) true jika P1>P2
membandingkan 2 pecahan, apakah P1 > P2: n1/d1 > n2/d2
*jika dan hanya jika n1*d2 > n2*d1}*

AddP(n1,d1,n2,d2): n-baru= (n1*d2 + n2*d1)
d-baru= (d1*d2)

REALISASI

AddP(P1,P2): MakeP(
(Pemb(P1)*Peny(P2) + Pemb(P2)*Peny(P1)),
(Peny(P1)*Peny(P2))
)

Konstruktor untuk membuat Pecahan baru sebagai hasil pertambahan

Penyebut baru

Pembilang baru

SubP(P1,P2): MakeP(
(Pemb(P1)*Peny(P2) - Pemb(P2)*Peny(P1)),
(Peny(P1)*Peny(P2))
)

MulP(P1,P2): MakeP(
(Pemb(P1)*Pemb(P2)),
(Peny(P1)*Peny(P2))
)

DivP(P1,P2): MakeP(
(Pemb(P1)*Peny(P2)),
(Peny(P1)*Pemb(P2))
)

REALISASI

RealP(P): Pemb(P)/Peny(P)

IsEqP?(P1,P2):
Pemb(P1)*Peny(P2) = Peny(P1) * Pemb(P2)

IsLtP?(P1,P2):
Pemb(P1)*Peny(P2) < Peny(P1) * Pemb(P2)

IsGtP?(P1,P2):
Pemb(P1)*Peny(P2) > Peny(P1) * Pemb(P2)

Type Date

- Diktat Tipe Bentukan hlm. 6-9
  - Definisi type Date
    - Penggunaan type bentukan lain
  - Definisi dan spesifikasi Selektor
  - Definisi dan spesifikasi Konstruktor
  - Definisi, spesifikasi, realisasi Predikat
    - Contoh: IsBefore? (versi dengan let dan tanpa let)
  - Definisi, spesifikasi, realisasi Fungsi Lain
    - Contoh: Nextday

TYPE DATE

DEFINISI DAN SPESIFIKASI TYPE

type Hr: integer [1..31]
{definisi ini hanyalah untuk "menamakan" type integer dgn nilai tertentu supaya mewakili hari, shg jika dipunyai suatu nilai integer, kita dapat memeriksa apakah nilai integer tersebut mewakili Hari yg absah}
type Bln: integer [1..12]
{definisi ini hanyalah untuk "menamakan" type integer dgn nilai tertentu supaya mewakili bulan}
type Thn: integer > 0
{definisi ini hanyalah untuk "menamakan" type integer dgn nilai tertentu supaya mewakili tahun}
type date <d: Hr, m: Bln, y: Thn>
{<d,m,y> adalah tanggal d bulan m tahun y}

DEFINISI DAN SPESIFIKASI SELEKTOR

Day: date → Hr
{Day(D) memberikan hari d dari D yang terdiri dari <d,m,y>}
Month: date → Bln
{Month(D) memberikan bulan m dari D yang terdiri dari <d,m,y>}
Year: date → Thn
{Year(D) memberikan tahun y dari D yang terdiri dari <d,m,y>}

DEFINISI DAN SPESIFIKASI KONSTRUKTOR

MakeDate: <Hr,Bln,Thn> → date

{MakeDate(h,b,t) membentuk tanggal pada hari, bulan, tahun tertentu}

DEFINISI DAN SPESIFIKASI OPERATOR TERHADAP DATE

NextDate: date → date

{NextDate(D): menghitung date yg merupakan keesokan hari dari date D yg diberikan, misal NextDate(<30,4,1980>) adalah <1,5,1980>}

Yesterday: date → date

{Yesterday(D): menghitung date yg merupakan 1 hari sebelum date D yg diberikan, misal Yesterday(<1,3,1980>) adalah <29,2,1980>}

NextNDay: date, integer → date

{NextNDay(D): menghitung date yg merupakan hari ke-N dari date D yg diberikan, misal NextNday(<30,4,1980>,3) adalah <3,5,1980>}

HariKe: date → integer[0..366]

{HariKeDlmThn(D): menghitung jumlah hari terhadap 1 Januari pada tahun y dengan memperhitungkan apakah y adalah tahun kabisat atau bukan}

DEFINISI DAN SPESIFIKASI PREDIKAT

IsEqD?: 2 date → boolean
{IsEqD?(D1,D2) true jika D1=D2 yaitu d1=d2, m1=m2 dan y1=y2}

IsBefore?: 2 date → boolean
{IsBefore?(D1,D2) true jika D1 adalah sebelum D2}

IsAfter?: 2 date → boolean
{IsAfter?(D1,D2) true jika D1 adalah sesudah D2}

IsKabisat?: Thn → boolean
{IsKabisat?(a) true jika tahun a adalah tahun kabisat: habis dibagi 4 tetapi tidak habis dibagi 100 atau habis dibagi 400}

REALISASI

{hanya sebagian, sisanya silakan dibuat sebagai latihan}

IsBefore?(D1,D2): periksa tahunnya, jika sama maka periksa bulannya, jika sama maka periksa tanggalnya

```
IsBefore?(D1,D2):
Let J1=Day(D1), M1=Month(D1), T1=Year(D1),
    J2=Day(D2), M2=Month(D2), T2=Year(D2)
in
    if T1 ≠ T2 then
        T1 < T2
    else
        if  M1 ≠ M2 then
            M1 < M2
        else
            J1 < J2
```

```
{Versi lain (Tanpa Let)}
IsBefore?(D1,D2):
  if Year(D1) ≠ Year(D2) then
      Year(D1) < Year(D2)
  else
      if  Month(D1) ≠ Month(D2) then
          Month(D1) < Month(D2)
      else
          Day(D1) < Day(D2)
```

REALISASI

IsEqD?(D1,D2): Year(D1) = Year(D2) and then
HariKeDlmThn(D1) = HariKeDlmThn(D2)

IsBefore?(D1,D2): if (Year(D1) = Year(D2)) then
HariKeDlmThn(D1) < HariKeDlmThn(D2)
else
Year(D1) < Year(D2)

IsAfter?(D1,D2): if (Year(D1) = Year(D2)) then
HariKeDlmThn(D1) > HariKeDlmThn(D2)
else
Year(D1) > Year(D2)

IsKabisat?(a):
((a mod 4 = 0) and (a mod 100 ≠ 0))
or
(a div 400 = 0)

Type Bentukan digunakan dalam Type Bentukan Lain

- Contoh: Type Garis
  - Suatu Garis terdiri atas 2 buah Point

```
type Point : < x : real , y : real >
    { <x,y> adalah sebuah point/titik, dengan x adalah absis dan y
      adalah ordinat }

type Garis : < PAw : Point , PAkh : Point >
    { <PAw, PAkh> adalah sebuah garis dengan PAw adalah titik
      awal garis dan PAkh adalah titik akhir garis }
```

Type Bentukan tanpa nama

- Fungsi dapat mengembalikan range type bentukan tanpa nama
- Contoh: EkivalensiDetik (diktat Tipe Bentukan hlm. 10)
  - Versi dengan let dan fungsi antara
  - Versi dengan kalkulasi tanpa let dan fungsi antara:

```
HHMMDD (x): < x div 86400,
              (x mod 86400) div 3600,
              ((x mod 86400) mod 3600) div 60,
              ((x mod 86400) mod 3600) mod 60 >
```

Translasi ke Haskell

Definisi Type

- Type didefinisikan menggunakan keyword data

```
-- Definisi Type
data Point = Pt Float Float
```

- Sebenarnya Pt merupakan konstruktor, yang dijadikan *internal representation* dari sebuah type bentukan Point yang menyimpan dua buah nilai bertipe Float
- Tambahan: deriving (Show) ditambahkan agar type dapat ditampilkan sebagai string melalui fungsi print nanti

```
-- Definisi Type
data Point = Pt Float Float deriving (Show)
```

Konstruktor

- Konstruktor direalisasikan sebagai fungsi untuk membentuk suatu type bentukan

```
-- Konstruktor
makePoint :: Float -> Float -> Point
makePoint x y = Pt x y
```

- Fungsi ini berperan sebagai *wrapper* dari konstruktor Pt

Selektor

- Selektor direalisasikan sebagai fungsi untuk mengakses komponen-komponen type bentukan

```
-- Selektor
absis :: Point -> Float
absis (Pt x _) = x

ordinat :: Point -> Float
ordinat (Pt _ y) = y
```

Predikat

- Merupakan fungsi yang menghasilkan nilai boolean

```
isOrigin :: Point -> Bool
isOrigin p = absis p == 0 && ordinat p == 0
```

Fungsi Lainnya

- Fungsi-fungsi lain yang memanfaatkan primitif dari type bentukan

```
jarak :: Point -> Point -> Float
jarak p1 p2 = sqrt((absis p1 - absis p2)^2 + (ordinat p1 - ordinat p2)^2)

jarak0 :: Point -> Float
jarak0 p = sqrt ((absis p)^2 + (ordinat p)^2)

kuadran :: Point -> Int
kuadran p
    | absis p > 0 && ordinat p > 0 = 1
    | absis p < 0 && ordinat p > 0 = 2
    | absis p < 0 && ordinat p < 0 = 3
    | absis p > 0 && ordinat p < 0 = 4

movePoint :: Float -> Float -> Point -> Point
movePoint dx dy p = makePoint (absis p + dx) (ordinat p + dy)
```

Aplikasi Type Bentukan

```
main :: IO ()
main = do
    let p1 = makePoint 5.0 12.0
    print (absis p1)                    -- 5.0
    print (ordinat p1)                  -- 12.0
    print (isOrigin p1)                 -- False
    print (movePoint 1.0 (-2.0) p1)     -- Pt 6.0 10.0
    print (jarak0 p1)                    -- 13.0
    let p2 = makePoint 10.0 24.0
    print (jarak p1 p2)                 -- 13.0
    let p3 = makePoint (-10.0) (-20.0)
    print (kuadran p1)                  -- 1
    print (kuadran p3)                  -- 3
```

Type bentukan tanpa nama

- Fungsi yang menghasilkan range type bentukan tanpa nama direalisasikan sebagai fungsi yang menghasilkan pasangan nilai (tuple)

```
hhmmdd :: Int -> (Int, Int, Int, Int)
hhmmdd x =
    (div x 86400,
    div (mod x 86400) 3600,
    div (mod (mod x 86400) 3600) 60,
    mod (mod (mod x 86400) 3600) 60)
```

Type bentukan tanpa nama

- Contoh: Aplikasi fungsi HHMMDD dari program EkivalensiDetik (diktat hlm.44)

```
⇒(hhmmdd 93660)
(1, 2, 1, 0)
⇒(hhmmdd 43200)
(0, 12, 0, 0)
```

Sebagai latihan, Anda dapat mengerjakan latihan soal di dalam diktat Tipe Bentukan hlm. 11.